New Generation Data Models and DBMSs Course - Assignment 3 - NoSQL Queries

Carlo Falchi (Matr. 67340A)[1*]
and Francesco Palladini (Matr. 85652A)[1*]

[1]Department of Computer Science, University of Milan, Italy.

*Corresponding author(s). E-mail(s): carlo.falchi@studenti.unimi.it;
francesco.palladini@studenti.unimi.it;

1 Assignment

1. For each university, identify the name, kind and city of the university that are located in California or Massachusetts along with the name, cfu, teacher surname and number of participants of the courses that it offers.
2. For each university, identify the teachers that teach the top three courses in terms of number of participants.
3. Identify the students that passed all their exams for courses with a written text with an average mark above 27.
4. For each course, report the list of students that participated to the course. Report also the number of students that passed the course with a mark between 18 and 23, between 24 and 27, and above 27. Report also the number of students that did not pass the course.
5. Identify the pair of students that are friend of a friend (i.e. a student should be associated with the friends of his friends).

1.1 First Query

For each university, identify the name, kind and city of the university that are located in California or Massachusetts along with the name, cfu, teacher surname and number of participants of the courses that it offers.

(a) Explanation

```
db.university.aggregate([
    { "$match":
        { "state": { "$in": ["California", "Massachusetts"] } }
    },
    {"$lookup": {
      "from": "course", "localField": "_id", "foreignField": "
          university", "as": "courses",
      "pipeline": [{ "$project": { "_id": 0, "name": 1, "cfu": 1,
          "teacher": "$teacher.surname", "numPartecipants": { "
          $size": "$participants" } } }]
    }},
    {"$project":
        { "_id": 0, "name": 1, "kind": 1, "city": 1, "courses": 1
            }
    }
])
```

1.2 Second Query

For each university, identify the teachers that teach the top three courses in terms of number of participants.

(a) Explanation

```
db.course.aggregate([
  { "$addFields": { "numParticipants": { "$size": "$participants"
      } } },
  { "$group": { "_id": { "univ": "$university", "name": "$teacher
      .name", "surname": "$teacher.surname" }, "maxParticipants":
      { "$max": "$numParticipants" } } },
  { "$sort": { "maxParticipants": -1 } },
  { "$group": { "_id": "$_id.univ", "teachers": { "$firstN": { "
      input": { "name": "$_id.name", "surname": "$_id.surname" },
      "n": 3 } } } },
  { "$lookup": { "from": "university", "localField": "_id", "
      foreignField": "_id", "as": "univ" } },
  { "$unwind": { "path": "$univ" } },
  { "$project": { "name": "$univ.name", "teachers": "$teachers" }
      }
])
```

1.3 Third Query

Identify the students that passed all their exams for courses with a written text with an average mark above 27.

(a) Explanation

```
db.student.aggregate([
  { $unwind: "$exams" },
  {
    $lookup: {
      from: "course",
      localField: "exams.course",
      foreignField: "_id",
      as: "course",
      pipeline: [{ $project: { exam_type: 1 } }]
    }
  },
  { $match: { "course.exam_type": "Written" } },
  {
    $group: {
      _id: "$_id",
      average: { $avg: "$exams.mark" },
      name: { $first: "$name" },
      surname: { $first: "$surname" }
    }
  },
  { $match: { average: { $gt: 27 } } },
  { $project: { _id: 0, average: 0 } }
])
```

1.4 Fourth Query

For each course, report the list of students that participated to the course. Report also the number of students that passed the course with a mark between 18 and 23, between 24 and 27, and above 27. Report also the number of students that did not pass the course.

(a) Explanation

```
db.course.aggregate([
  { $unwind: "$participants" },
  {
    $lookup: {
      from: "student",
      localField: "participants",
      foreignField: "_id",
      as: "student"
    }
  },
  { $unwind: "$student" },
  {
    $addFields: {
      currentCourseExam: {
```

```
        $first: {
          $filter: {
            input: "$student.exams",
            as: "exam",
            cond: { $eq: ["$$exam.course", "$_id"] }
          }
        }
      }
    }
  },
  {
    $addFields: {
      didntPassExam: {
        $cond: [{ $not: ["$currentCourseExam"] }, 1, 0]
      },
      passedwith18to23: {
        $cond: [
          {
            $and: [
              { $gte: ["$currentCourseExam.mark", 18] },
              { $lte: ["$currentCourseExam.mark", 23] }
            ]
          },
          1,
          0
        ]
      },
      passedwith24to27: {
        $cond: [
          {
            $and: [
              { $gte: ["$currentCourseExam.mark", 24] },
              { $lte: ["$currentCourseExam.mark", 27] }
            ]
          },
          1,
          0
        ]
      },
      passedabove27: {
        $cond: [{ $gt: ["$currentCourseExam.mark", 27] }, 1, 0]
      }
    }
  },
  {
    $group: {
      _id: "$_id",
      name: { $first: "$name" },
      participants: { $push: "$student" },
      total_passedwith18to23: { $sum: "$passedwith18to23" },
```

```
      total_passedwith24to27: { $sum: "$passedwith24to27" },
      total_passedabove27: { $sum: "$passedabove27" },
      total_didntPassExam: { $sum: "$didntPassExam" }
    }
  }
])
```

1.5 Fifth Query

Identify the pair of students that are friend of a friend (i.e. a student should be associated with the friends of his friends).

(a) Explanation

```
db.student.aggregate(
[
    { $unwind: "$friends" },
    {
        $lookup: { from: "student", localField: "friends",
            foreignField: "_id", as: "friend_info" }
    },
    { $unwind: "$friend_info" },
    { $unwind: "$friend_info.friends" },
    {
        $group: {
            _id: { _id: "$_id", friendOfFriendId: "$friend_info.
                friends" },
            name: { $first: "$name" },
            surname: { $first: "$surname" }
        }
    },
    {
        $lookup: { from: "student", localField: "_id.
            friendOfFriendId", foreignField: "_id", as: "
            friendoffriend" }
    },
    { $unwind: "$friendoffriend" },
    { $project: { _id: 0, name: 1, surname: 1, friendofFriendName
        : "$friendoffriend.name", friendofFriendSurname: "
        $friendoffriend.surname" } }
])
```